



Máster en Cloud Apps

Desarrollo y despliegue de aplicaciones en la nube

Curso académico 2022/2023

Trabajo de Fin de Máster

Crossfit box app

Autor: Adrian Revilla

Tutor: Patxi Gortazar



Introducción y objetivos

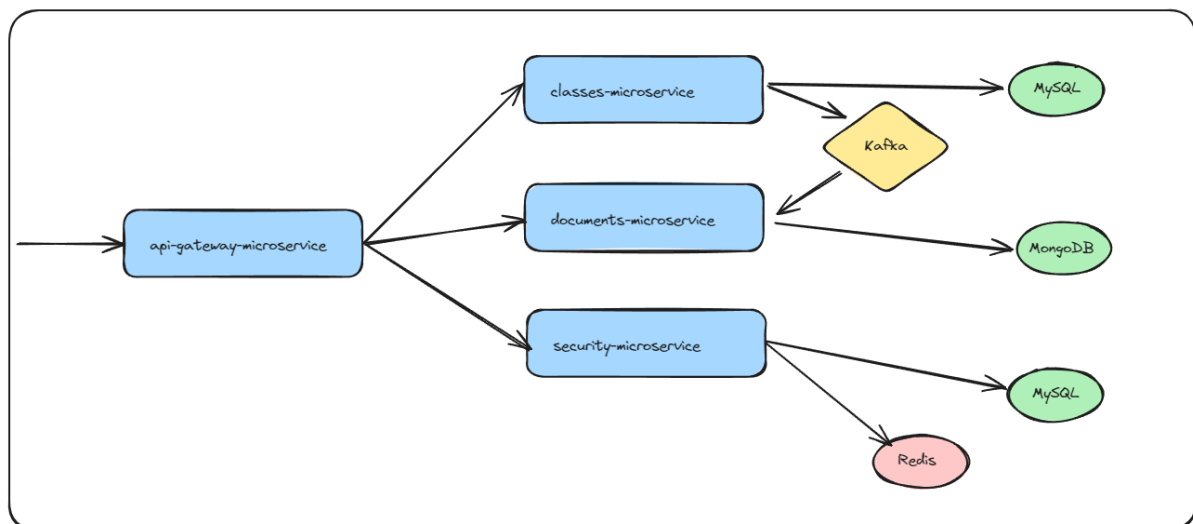
El proyecto consiste en desarrollar una aplicación que engloba varias fases del ciclo de vida del software, como es el diseño, el desarrollo en sí, el testing, el despliegue y el mantenimiento.

La temática data de un gestor de reservas de una aplicación para un box de crossfit, en la cual el usuario podrá registrarse, identificarse, reservar clases, consultar clases y descargar la memoria de su historial.

El objetivo principal es definir una aplicación funcional donde el usuario pueda operar a su antojo, realizando varias operaciones contra el backend que va a componer la aplicación.

Arquitectura

En cuanto a la arquitectura, va a ser una aplicación con diferentes microservicios divididos por alcance, en la que constarán un gateway como puerta de entrada, un micro para la parte de seguridad, otro para la parte de gestión de reservas y otro para el almacenamiento de documentos.



En cuanto a los diferentes componentes que componen la arquitectura, se definirá una redis como base de datos en memoria para almacenar sesiones, un kafka para la transferencia de datos de manera asíncrona y varias bases de datos sql y nosql dependiendo de la necesidad en cada situación, además de volúmenes de persistencia para almacenamiento de datos.

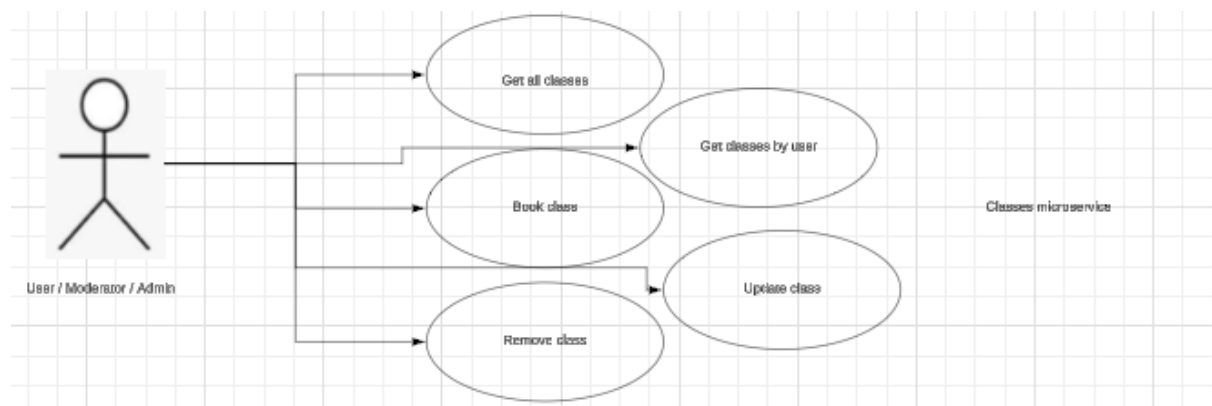
Tendremos un cluster compuesto por los diferentes elementos mencionados y dividido por namespaces para lo relacionado con los componentes per se, la parte de monitorización, el sistema de CD y la parte de testing.

El sistema estará dividido en dos entornos, desarrollo y producción, en los cuales se desplegará la infraestructura en base a las acciones realizadas contra el repositorio github. Los commits realizados sobre la rama development, lanzarán el sistema de CI y el CD desplegará los cambios en el cluster minikube local. En cambio, los commits realizados sobre master (via PR) lanzarán las acciones terraform pertinentes para levantar el cluster en AWS.

Análisis

Esta sección trata de mostrar los diferentes diagramas de los casos de uso recopilados a lo largo del proyecto para cada microservicio.

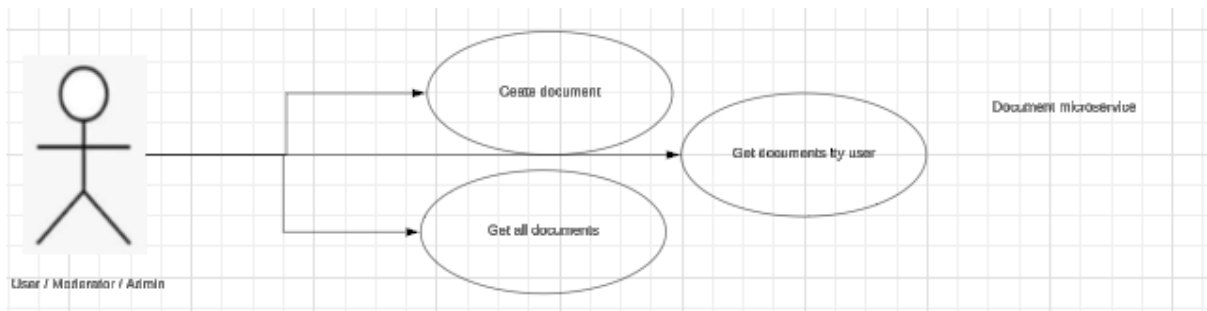
Microservicio para gestionar las clases



Classes microservice

GET /api/classes	Get all classes
POST /api/classes/user/20895705K	Book a class
DELETE /api/classes/user/20895705K	Remove from a class
POST /api/classes/class/1/user/20895705K	Update user from class
GET /api/classes/user/20895705K	Get user classes

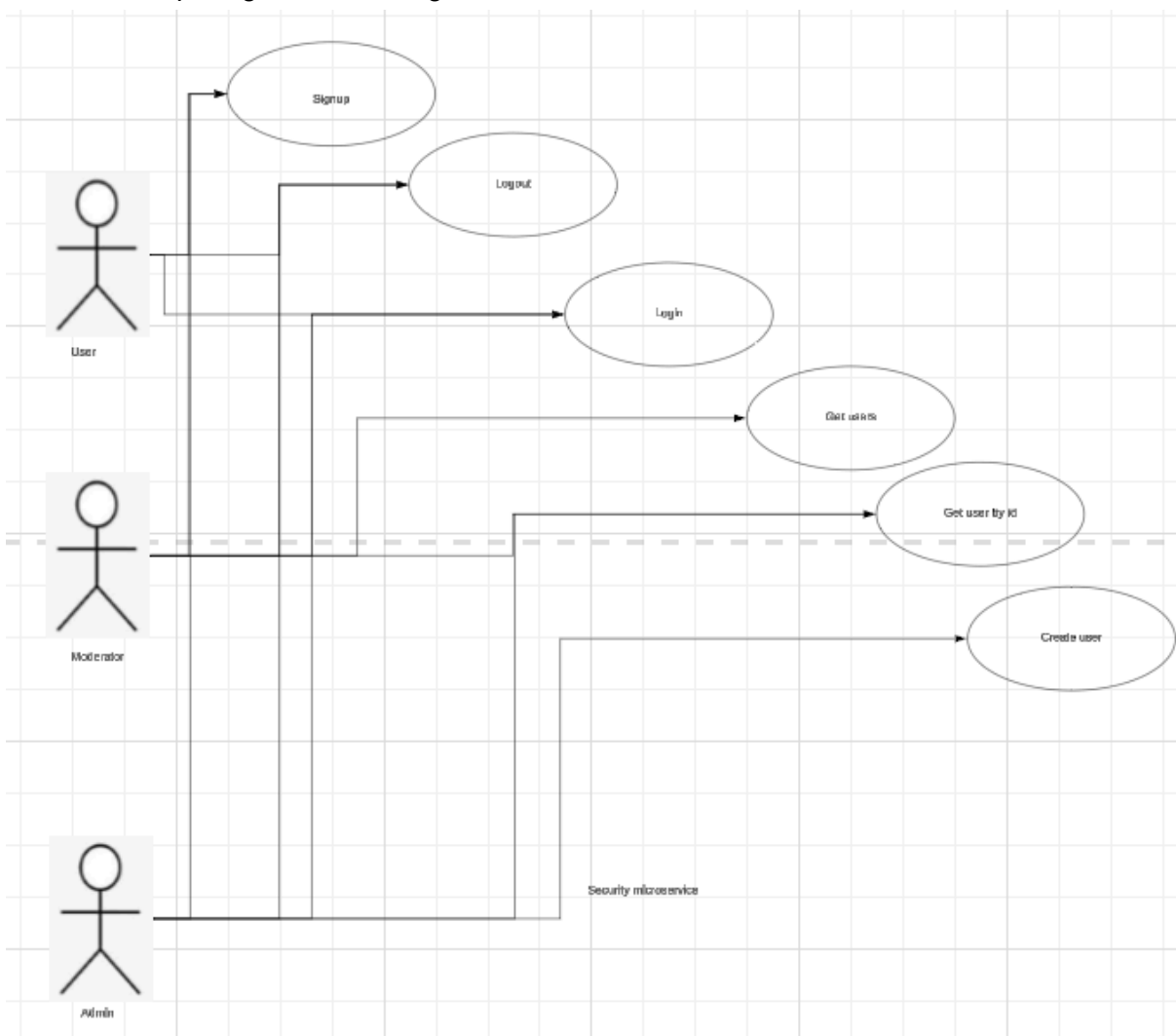
Microservicio para gestionar los documentos



Document microservice

- POST /api/doduments/user/ 20895705K Create document
- GET /api/documents Get all documents
- GET /api/documents/user/ 20895705K Get documents by user

Microservicio para gestionar la seguridad



Security microservice

POST /api/auth/signup	Signup
POST /api/auth/signin	Signin
GET /api/users	Get all users
GET /api/user/1	Get user by id
POST /api/user	Create user
POST /api/auth/logout	Logout

Desarrollo

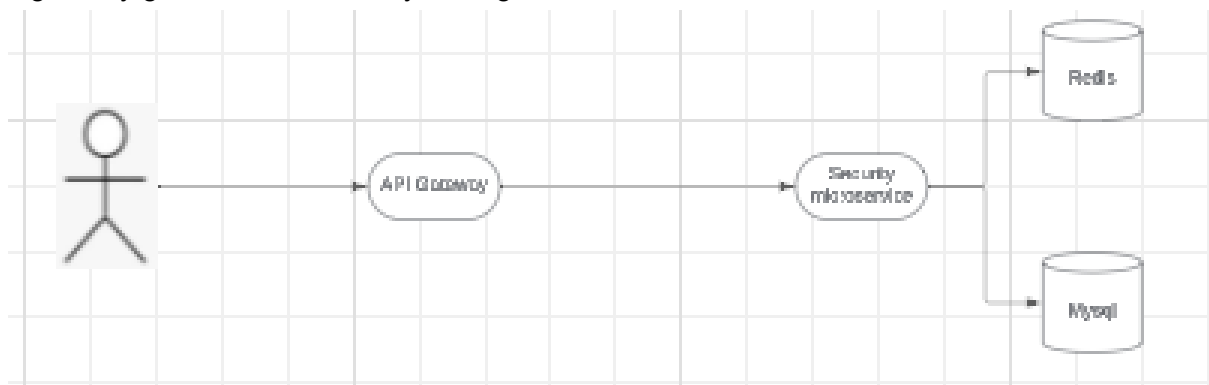
Se va a dividir en secciones las diferentes situaciones abordadas durante el desarrollo. Las secciones mencionadas van a ser las siguientes:

- Aplicación
- Orquestación
- Integración continua
- Monitorización
- Despliegue continuo
- Testing
- Mantenimiento

Aplicación

En esta sección se van a detallar los diferentes microservicios mencionados a nivel técnico.

El primero de los microservicios trata de la aplicación de seguridad, que tiene el objetivo de registrar y gestionar el acceso y deslogueo de los usuarios.



Este microservicio está estrechamente ligado a la caché redis ya que trata de almacenar sesiones y eliminarlas, para así poder identificar fácilmente a los usuarios que quieran

acceder a los otros microservicios. Se utiliza lettuce como librería dada las ventajas de concurrencia y asincrónica que permite respecto a otras.

```

@Bean
public RedisConnectionFactory redisConnectionFactory() {
    RedisStandaloneConfiguration redisStandaloneConfiguration = new RedisStandaloneConfiguration();
    redisStandaloneConfiguration.setHostName(host);
    redisStandaloneConfiguration.setPort(port);
    logger.info("Redis Host: " + host);
    logger.info("Redis Port: " + port);

    LettuceClientConfiguration lettuceClientConfiguration = LettuceClientConfiguration.defaultConfiguration();

    return new LettuceConnectionFactory(redisStandaloneConfiguration, lettuceClientConfiguration);
}

//Creating RedisTemplate for Entity 'Session'
@Bean
public RedisTemplate<String, Session> redisTemplate(RedisConnectionFactory redisConnectionFactory) {
    RedisTemplate<String, Session> redisTemplate = new RedisTemplate<>();
    redisTemplate.setConnectionFactory(redisConnectionFactory);
    redisTemplate.setKeySerializer(new StringRedisSerializer());
    redisTemplate.setValueSerializer(new GenericJackson2JsonRedisSerializer());
    return redisTemplate;
}

```

```

@CrossOrigin(origins = "*", maxAge = 3600)
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private static final Logger logger = LoggerFactory.getLogger(AuthController.class);

    @Autowired
    AuthService authService;

    @PostMapping("/login")
    public ResponseEntity<?> authenticateUser(@Valid @RequestBody LoginRequest loginRequest) {
        logger.info("### authenticateUser ###");
        try {
            JwtResponse jwtResponse = this.authService.authenticateUser(loginRequest);
            return ResponseEntity.ok(jwtResponse);
        } catch (Exception e) {
            logger.error(String.valueOf(e.getStackTrace()));
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(e.getMessage());
        }
    }
}

```

Se utilizará Spring security para securizar la aplicación mediante tokens JWT y roles para verificar la autenticación y autorización de usuarios. Se definirá un filtro que determinará la veracidad del token por cada request recibida en el microservicio.

```

@Service
public class RedisServiceImpl implements RedisService {

    private static final Logger logger = LoggerFactory.getLogger(RedisServiceImpl.class);

    @Autowired
    @Qualifier("sessionDaoImpl")
    ISessionDao sessionDao;

    public void saveData(String email, String token) {
        try {
            sessionDao.saveSession(new Session((int)(Math.random()*10000000+1),email, token));
        } catch (Exception e) {
            logger.error(e.getMessage());
        }
    }
}

```

```

@CrossOrigin(origins = "*", maxAge = 3600)
@RestController
@RequestMapping("/api/users")
public class UserRestController {

    private static final Logger logger = LoggerFactory.getLogger(UserRestController.class);

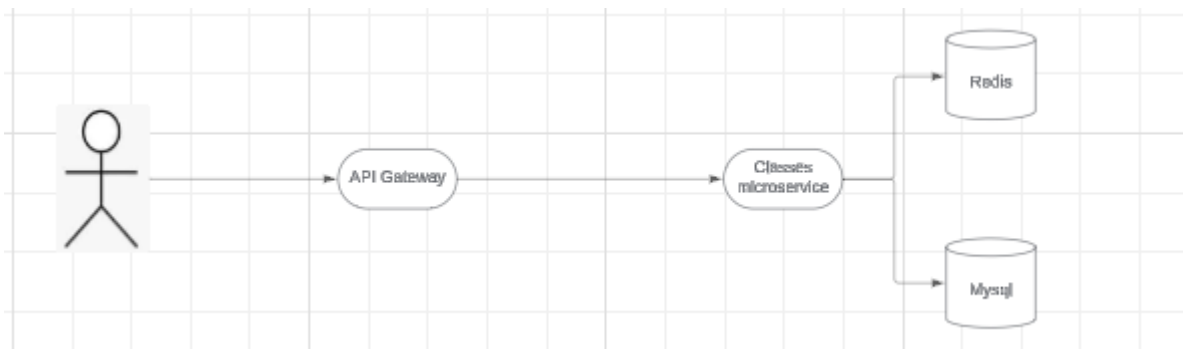
    private final UserService userService;

    public UserRestController(UserService userService) {
        this.userService = userService;
    }

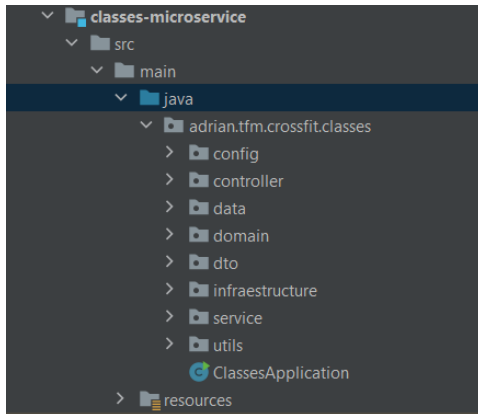
    @Operation(summary = "Get all users")
    @ApiResponse(responseCode = "200", description = "users found",
        content = { @Content(mediaType = "application/json",
            array = @ArraySchema(schema = @Schema(implementation = UserDTO.class))) })
    @GetMapping("/")
    @PreAuthorize("hasRole('MODERATOR') or hasRole('ADMIN')")
    public ResponseEntity<?> getUsers(Pageable pageable) {
        logger.info("### getUsers ###");
        try {
            return ResponseEntity.ok(userService.findAll(pageable));
        } catch (Exception e) {
            logger.error(String.valueOf(e.getStackTrace()));
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(e.getMessage());
        }
    }
}

```

El segundo de los microservicios trata de la aplicación de gestión de datos que está estrechamente ligada con el kafka y una base de datos MySQL. El desarrollo de esta aplicación se ha realizado utilizando la arquitectura hexagonal para abstraer la implementación de las dependencias y darle versatilidad.



Además, como cualquiera de los otros microservicios, dependerá de la redis mencionada anteriormente para verificar la identidad de los usuarios en cada petición. Para ello, se genera un interceptor que verifica la sesión mediante el token JWT utilizado.



```
public class SessionInterceptor implements HandlerInterceptor {
    2 usages
    private static final Logger logger = LoggerFactory.getLogger(SessionInterceptor.class);

    3 usages
    @Autowired
    @Qualifier("sessionDaoImpl")
    ISessionDao sessionDao;

    1 adrianrevilla009
    public SessionInterceptor(@Qualifier("sessionDaoImpl") ISessionDao sessionDao) { this.sessionDao = sessionDao; }

    1 adrianrevilla009
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler)
        throws Exception {
        String token;
        try {
            token = JwtUtils.parseJwt(request);
        } catch (Exception e) {
            throw new Exception(e.getMessage());
        }
        logger.info(sessionDao.toString());
    }
}
```

Se desarrollan topics en el emisor y receptor de la cola kafka para el envío de datos de manera asincrona.

```
@Override
@KafkaListener(topics = "get-classes-topic", groupId = "classes-group")
public void receiveGetClassesByNifMessage(ClassesRequestMessageDto classesRequestMessageDto) throws Exception {
    logger.info("Task status is updated : " + classesRequestMessageDto.getNif());

    List<ClassEntity> classEntityList = this.classJpaRepository.findByUserNif(classesRequestMessageDto.getNif());

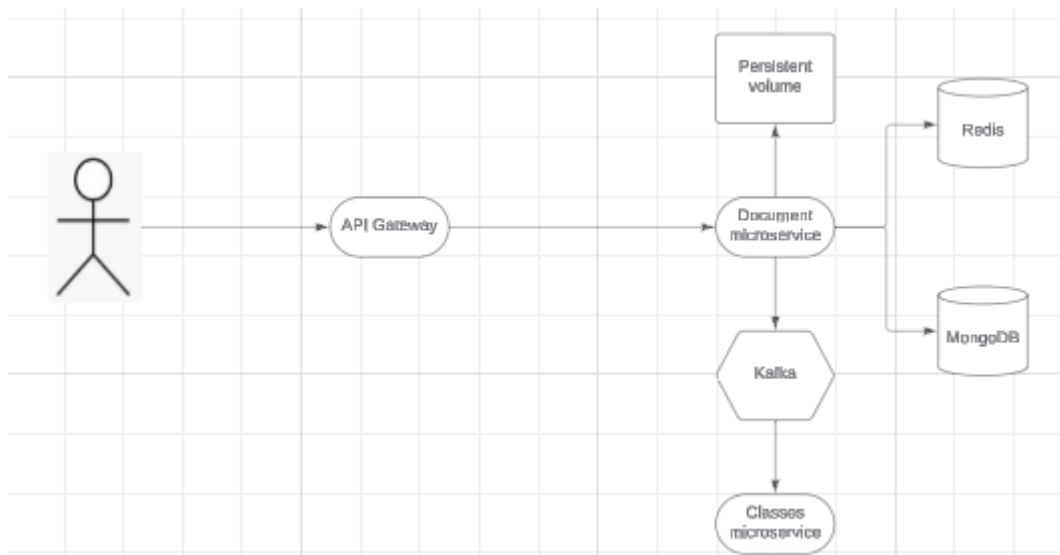
    this.sendGetClassesByNifMessage(topicName: "send-classes-topic",
        classEntityList.stream().map(c -> this.classDtoAndEntityMapper.mapFromClassEntityToClassDto(c)).collect(Collectors.toList());
        classesRequestMessageDto.getNif()
    );
}
```

También se genera una colección de datos en la base de datos relacional para que los usuarios puedan consumirlos desde fuera y darle realismo a la aplicación.

```
@PostConstruct
@Transactional
public void init() throws ParseException {
    logger.info("INITIALIZING DB DATA");

    ExerciseDto exerciseDto1 = new ExerciseDto(id: 1L, name: "DUMBBELL WALKING LUNGE");
    ExerciseDto exerciseDto2 = new ExerciseDto(id: 2L, name: "BENT-OVER REAR-DELT PLATE RAISE");
    ExerciseDto exerciseDto3 = new ExerciseDto(id: 3L, name: "RUNNING SPRINT");
    ExerciseDto exerciseDto4 = new ExerciseDto(id: 4L, name: "TOES TO BAR");
    ExerciseDto exerciseDto5 = new ExerciseDto(id: 5L, name: "OVERHEAD DUMBBELL REVERSE LUNGE");
}
```

El tercero de los microservicios sirve para el almacenamiento de documentos, que actúa como un gestor de los mismos mediante un volumen de persistencia y una base de datos no sql que almacenará los metadatos de los documentos como archivos json y los documentos per se, en un volumen como mencionaba.



El micro está estrechamente ligado con el recién mencionado para obtener los datos de las clases de crossfit desde una cola de mensajería kafka y así poblar los documentos.

```

@Repository
public interface DocumentRepository extends MongoRepository<Document, String> {
    List<Document> findAll();

    @Query("{ 'user.nif': ?0 }")
    List<Document> findByUser(String nif);
}

@Override
public void sendGetClassesByNifMessage(String topicName, String nif) {
    var future = CompletableFuture.supplyAsync(() -> kafkaTemplate.send(topicName, nif));

    future.whenComplete((sendResult, exception) -> {
        if (exception != null) {
            future.completeExceptionally(exception);
        } else {
            future.complete(sendResult);
        }
    });

    logger.info("Task status send to kafka topic : " + nif);
}

```

Este almacenamiento hasta la fecha se ha montado como una persistencia local para así almacenarlos en el propio contenedor docker, a falta de un bucket de persistencia S3 alojado en amazon la cual hubiera sido la opción adecuada.

```

@Override
public void createFile(String nif, List<ClassDto> classDtoList) throws Exception {
    if (!classDtoList.isEmpty()) {
        // create excel and save on mongo db
        Document excelFile = this.classesExcelService.createExcel(nif, classDtoList);
        // save it into a local persistence unit
        this.saveDocumentLocally(excelFile);
    } else {
        logger.info("[CREATE EXCEL] Excel not created due to empty data for user");
    }
}

```

El cuarto trata de un gateway desarrollado con spring gateway para tener más trazabilidad y operacionalidad de las peticiones entrantes frente a un ingress kubernetes. Se ha implementado con spring-gateway y hace de proxy inverso para redireccionar peticiones a los microservicios correspondientes en base a el context path de la petición. Además, se han generado configuraciones de seguridad e interceptores para logear toda petición entrante y saliente del gateway y luego poder trazar estos logs con el stack grafana para el monitoreo del cluster.


```

@Bean
public RouteLocator securityProxyRouting(RouteLocatorBuilder builder) {

    URI backendUri = URI.create(this.securityUrl);
    return builder.routes()
        .route(r -> r
            .path(_patterns: "/api/auth/**") BooleanSpec)
            .filters(f -> f
                .prefixPath(backendUri.getPath())
                .rewriteLocationResponseHeader(NEVER_STRIP.name(),
                    LOCATION_HEADER,
                    hostValue: null,
                    protocolsRegex: "http|https")
                .rewriteResponseHeader(LOCATION_HEADER, backendUri.getPath(),
                    hostValue: null,
                    protocolsRegex: "http|https")
                .filter(new RequestLoggingFilter())
                .filter(new ResponseLoggingFilter())) UnSpec)
            .uri(this.securityUrl))
        .route(r -> r
            .path(_patterns: "/api/user/**") BooleanSpec)
            .filters(f -> f
                .prefixPath(backendUri.getPath())
                .rewriteLocationResponseHeader(NEVER_STRIP.name(),
                    LOCATION_HEADER,
                    hostValue: null,
                    protocolsRegex: "http|https")
                .rewriteResponseHeader(LOCATION_HEADER, backendUri.getPath(),
                    hostValue: null,
                    protocolsRegex: "http|https")
                .filter(new RequestLoggingFilter())
                .filter(new ResponseLoggingFilter())) UnSpec)
            .uri(this.securityUrl))
        .build();
}

```

```

@Component
public class RequestLoggingFilter implements GatewayFilter {
    private static final Logger logger = LoggerFactory.getLogger(RequestLoggingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        logger.info("request: " + request.getMethod() + " " + request.getURI());
        return chain.filter(exchange);
    }
}

```

El último de los componentes es una librería que almacena clases comunes entre todos los microservicios y que sirve para evitar la duplicidad de código. Esta librería se almacena en un github artifact repository que va a ser utilizada por estas clases como una dependencia adicional.

Este sería el repositorio donde se almacena la librería cuando se ejecuta el CI mediante el manejador de distribuciones.

The screenshot shows a GitHub repository interface for 'common-TFM'. The left sidebar lists files and folders: .github/workflows, .mvn/wrapper, src, .gitignore, README.md, mvnw, mvnw.cmd, and pom.xml. The main area shows a commit history table with columns for file name, commit message, and time ago. On the right, the 'About' section is visible, showing 'Common packages for TFM Crossfit app'. Below this, the 'Packages' section is highlighted with a white arrow, showing 'adrian.tfm.crossfit.common'.

Y esta es la forma en la que los diferentes microservicios consiguen importarla desde el repositorio privado.

```

<!-- Commons own library -->
<dependency>
    <groupId>adrian.tfm.library</groupId>
    <artifactId>common</artifactId>
    <version>1.0.1-SNAPSHOT</version>
</dependency>

```

```

<plugin>
    <groupId>io.fabric8</groupId>
    <artifactId>docker-maven-plugin</artifactId>
    <version>0.35.0</version> <!-- Usa la última versión disponible -->
    <configuration>
        <images>
            <image>
                <name>adrian2606/common</name>
                <alias>common</alias>
                <run>
                    <namingStrategy>alias</namingStrategy>
                </run>
            </image>
        </images>
    </configuration>
</plugin>

```

Orquestación

En esta sección se va a detallar la forma de orquestar el cluster utilizando varios entornos y cuales han sido los diferentes componentes necesarios para hacerlo funcionar. Primero de todo, se ha definido un entorno de desarrollo con minikube y se ha definido otro para producción utilizando IAC con terraform y Amazon como proveedor.

La orquestación dentro de los clusters kubernetes se ha dividido por namespaces dependiendo del propósito de cada servicio levantado, como pueden ser el namespace de desarrollo, el de testing, el de cd, el de monitorización etc.

El entorno de desarrollo se compone de varios recursos que se han ido levantando mediante charts de helm para así poder montar los despliegues mediante releases. Para ello se han definido los manifiestos de los microservicios anteriormente mencionados, la cola de mensajería, las bases de datos y la redis junto con los volúmenes persistentes para así asegurar el mantenimiento de los datos en el tiempo.

Pods10 itemsNamespace: developmentSearch Pods...

☐	Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status	
☐	api-gateway-microservice-7db9...	development		0	ReplicaSet	minikube	Burstable	4d22h	Running	
☐	classes-microservice-59ccb7c4...	development		0	ReplicaSet	minikube	Burstable	4d22h	Running	
☐	documents-microservice-564c9...	development		0	ReplicaSet	minikube	Burstable	4d22h	Running	
☐	kafka-6b9fdd6655-qvvs	development		1	ReplicaSet	minikube	BestEffort	6d23h	Running	
☐	mongo-db-76d56c7b8c-j6szw	development		1	ReplicaSet	minikube	BestEffort	6d23h	Running	
☐	mysql-698595bd7f-wxxwt	development		1	ReplicaSet	minikube	BestEffort	6d23h	Running	
☐	mysql-db-security-5bf888b84b-j...	development		1	ReplicaSet	minikube	BestEffort	6d23h	Running	
☐	redis-cache-75b87df696-dm5q9	development		1	ReplicaSet	minikube	BestEffort	6d23h	Running	
☐	security-microservice-f57d484d...	development		0	ReplicaSet	minikube	Burstable	4d22h	Running	
☐	zookeeper-75f44db564-26fbs	development		1	ReplicaSet	minikube	BestEffort	6d23h	Running	

```
PS C:\Proyectos\master\TFM\crossfit-app-TFM> helm install development-cluster .\helm\
NAME: development-cluster
LAST DEPLOYED: Fri Jan 5 13:52:33 2024
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
PS C:\Proyectos\master\TFM\crossfit-app-TFM>
```

Estos son los recursos levantados para los diferentes namespaces, en caso del de testing se levanta un controlador y una serie de workers para tirar las pruebas de carga con jmeter.

Pods4 itemsNamespace: testingSearch Pods...

☐	Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status	
☐	jmeter-distributed-jmeter-maste...	testing		0	ReplicaSet	minikube	BestEffort	24h	Running	
☐	jmeter-distributed-jmeter-server...	testing		0	ReplicaSet	minikube	BestEffort	24h	Running	
☐	jmeter-distributed-jmeter-server...	testing		0	ReplicaSet	minikube	BestEffort	24h	Running	
☐	jmeter-distributed-jmeter-server...	testing		0	ReplicaSet	minikube	BestEffort	24h	Running	

En el caso del despliegue continuo, los recursos por defecto que levanta el argocd.

Pods 7 items									
Namespace: argocd									
Search Pods...									
☐	Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status
☐	argocd-application-controller-0	argocd		0	StatefulSet	minikube	BestEffort	75s	Running
☐	argocd-applicationset-controller...	argocd		0	ReplicaSet	minikube	BestEffort	76s	Running
☐	argocd-dex-server-75dff665f-x...	argocd		0	ReplicaSet	minikube	BestEffort	76s	Running
☐	argocd-notifications-controller-c...	argocd-notifications-controller-c8c5c4b-7bngx		0	ReplicaSet	minikube	BestEffort	76s	Running
☐	argocd-redis-6b8845f86d-4cm6b	argocd		0	ReplicaSet	minikube	BestEffort	76s	Running
☐	argocd-repo-server-7994cbdf5b...	argocd		0	ReplicaSet	minikube	BestEffort	76s	Running
☐	argocd-server-79b4969bc4-km9...	argocd		0	ReplicaSet	minikube	BestEffort	75s	Running

En el caso de la monitorización, los recursos por defecto que levanta el stack de grafana con prometheus y loki.

Pods 12 items									
Namespace: monitorizacion									
Search Pods...									
☐	Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status
☐	grafana-597b7497d-9h7z7	monitorizacion		0	ReplicaSet	minikube	BestEffort	14m	Running
☐	loki-0	monitorizacion		0	StatefulSet	minikube	BestEffort	94s	Running
☐	loki-canary-tqzzz	monitorizacion		0	DaemonSet	minikube	BestEffort	94s	Running
☐	loki-gateway-5f9c4b449c-ikzr9	monitorizacion		0	ReplicaSet	minikube	BestEffort	94s	Running
☐	loki-grafana-agent-operator-864...	monitorizacion		0	ReplicaSet	minikube	BestEffort	94s	Running
☐	loki-logs-2jq6s	monitorizacion		0	DaemonSet	minikube	BestEffort	11m	Terminati...
☐	loki-logs-8tqdp	monitorizacion		0	DaemonSet	minikube	BestEffort	92s	Running
☐	prometheus-alertmanager-0	monitorizacion		0	StatefulSet	minikube	BestEffort	15m	Running
☐	prometheus-kube-state-metrics...	monitorizacion		0	ReplicaSet	minikube	BestEffort	15m	Running
☐	prometheus-prometheus-node-e...	monitorizacion		0	DaemonSet	minikube	BestEffort	15m	Running
☐	prometheus-prometheus-pushq...	monitorizacion		0	ReplicaSet	minikube	BestEffort	15m	Running
☐	prometheus-server-799f594896...	monitorizacion		0	ReplicaSet	minikube	BestEffort	15m	Running

Integración continua

Se ha escogido github actions como sistema de integración continua para así poder realizar ciertas acciones sobre las ramas de trabajo una vez estas son enviadas al repositorio remoto. Se han definido dos flujos de trabajo sobre las ramas development y main.

Lo primero de todo, se definen unos dockerfiles para cada microservicio, y así poder generar las imágenes y subirlas a un repositorio remoto como es dockerhub.

```

1 FROM maven:3.9.0-eclipse-temurin-17 as builder
2 WORKDIR /crossfit-app-TFM/api-gateway-microservice
3 COPY /src /crossfit-app-TFM/api-gateway-microservice/src
4 COPY pom.xml /crossfit-app-TFM/api-gateway-microservice
5 RUN mvn -B clean package -DskipTests
6
7 # Imagen base para el contenedor de la aplicación
8 FROM eclipse-temurin:17-jdk
9 WORKDIR /usr/src/app/
10 COPY --from=builder /crossfit-app-TFM/api-gateway-microservice/target/*.jar /usr/src/app/
11
12 EXPOSE 8083
13 CMD [ "java", "-jar", "api-gateway-microservice-1.0.0.jar" ]

```

Para ello, se definen ciertos manifiestos que compilan, pasan la batería de test, generan la imagen, la taguean y la pushean al repositorio de imágenes mencionado.

Para poder gestionar el versionado de manera automática, se define un archivo .txt que incrementará la versión una vez las imágenes sean generadas, y así actualizarlas en los archivos helm para que tengan la última versión a desplegar.

```

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v1

- name: Login to Docker Hub
  uses: docker/login-action@v1
  with:
    username: ${ secrets.DOCKERHUB_USERNAME }
    password: ${ secrets.DOCKERHUB_TOKEN }

- name: Build and Push Docker Images
  run: |
    services=("api-gateway-microservice" "classes-microservice" "documents-microservice" "security-microservice")

    for service in "${services[@]}; do
      echo "Current directory: $(pwd)"
      ls -la
      cd $service
      echo "Changed to directory: $(pwd)"
      ls -la
      docker buildx create --use
      docker buildx inspect --bootstrap
      docker buildx build -f dockerfile.Dockerfile -t ${ secrets.DOCKERHUB_USERNAME }/$service:$VERSION --push .
      cd ..
    done

```

Aquí vemos la ejecución de un flujo con todos los steps definidos en la UI de github. Y la imagen generada en el repositorio de imágenes oficial de docker.

build-and-push

succeeded 5 days ago in 2m 41s

Beta

Give feedback

Q Search logs

> Set up job2s

> Pull devopsinfra/action-commit-pushv0.9.24s

> Checkout Repository3s

> Read and Increment Version0s

> Acceder a la Variable de Entorno0s

> Check if Skip CI0s

> Commit & Push changes1s

> Create Pull Request1s

> Set up Docker Buildx2s

> Login to Docker Hub0s

> Build and Push Docker Images2m 20s

> Post Login to Docker Hub0s

> Post Set up Docker Buildx0s

> Post Checkout Repository0s

> Complete job0s

adrian2606 / [Repositories](#) / [security-microservice](#) / [General](#) Using 0 of 1 private repositories. [Get more](#)

General

Tags

Builds

Collaborators

Webhooks

Settings

Add a short description for this repository

The short description is used to index your content on Docker Hub and in search engines. It's visible to users in search results.

Update

adrian2606/security-microservice

Updated 5 days ago

This repository does not have a description

Docker commands

To push a new tag to this repository:

`docker push adrian2606/security-microservice:tagname`

Public View

Tags

This repository contains 23 tag(s).

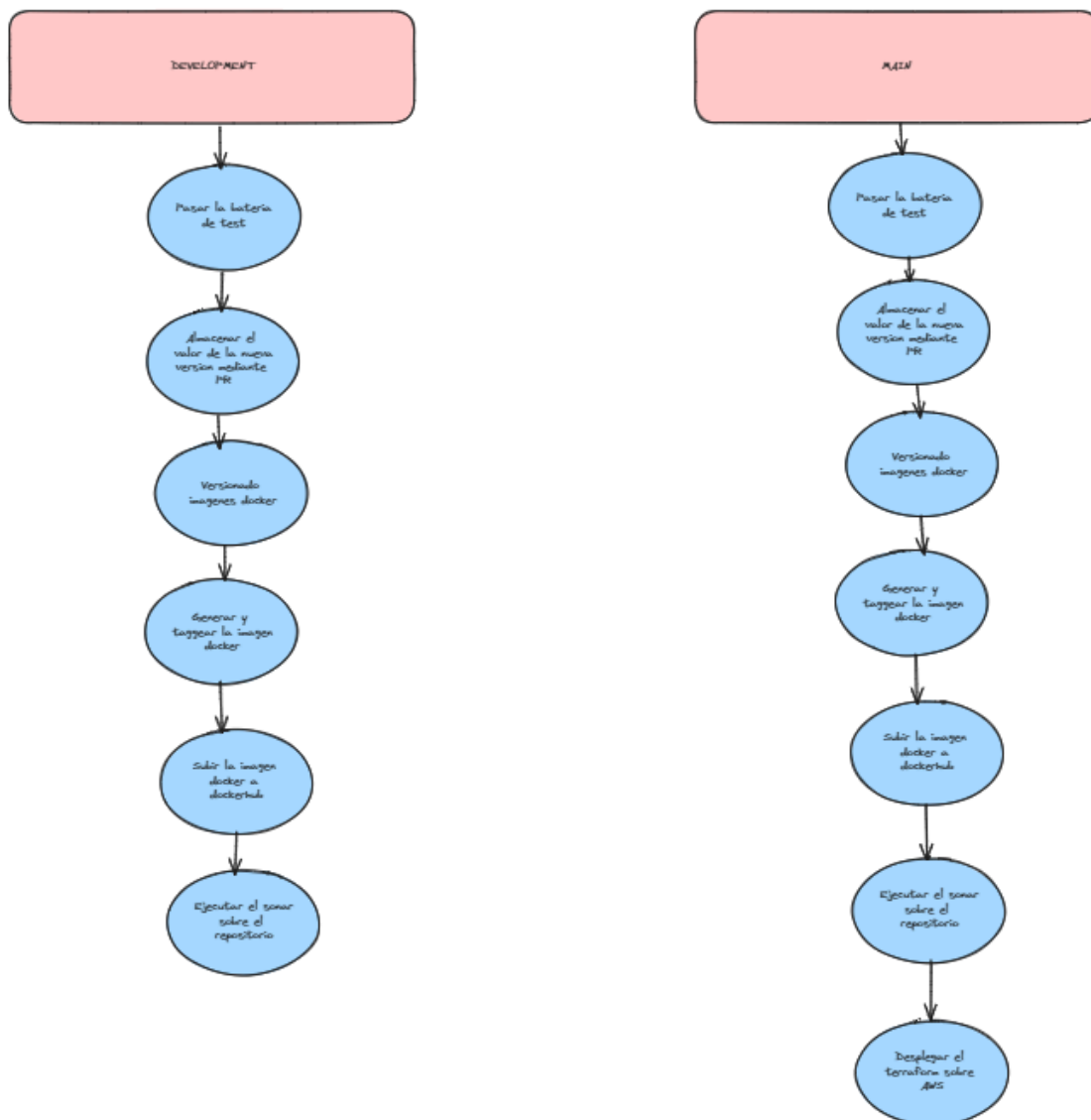
Tag	OS	Type	Pulled	Pushed
v32		Image	---	5 days ago

Automated Builds

Manually pushing images to Hub? Connect your account to GitHub or Bitbucket to automatically build and tag new images whenever your code is updated, so you can focus your time on creating.

Available with Pro, Team and Business subscriptions. [Read more about automated builds](#)

He aqui el diagrama de los pasos ejecutados en el github actions:



Despliegue continuo

La herramienta seleccionada ha sido argocd dada su fácil integración con kubernetes y github como repositorio de análisis de código. Esta herramienta permite comprobar cambios en el repositorio y una vez detectados, desplegarlos en el clúster seleccionado.

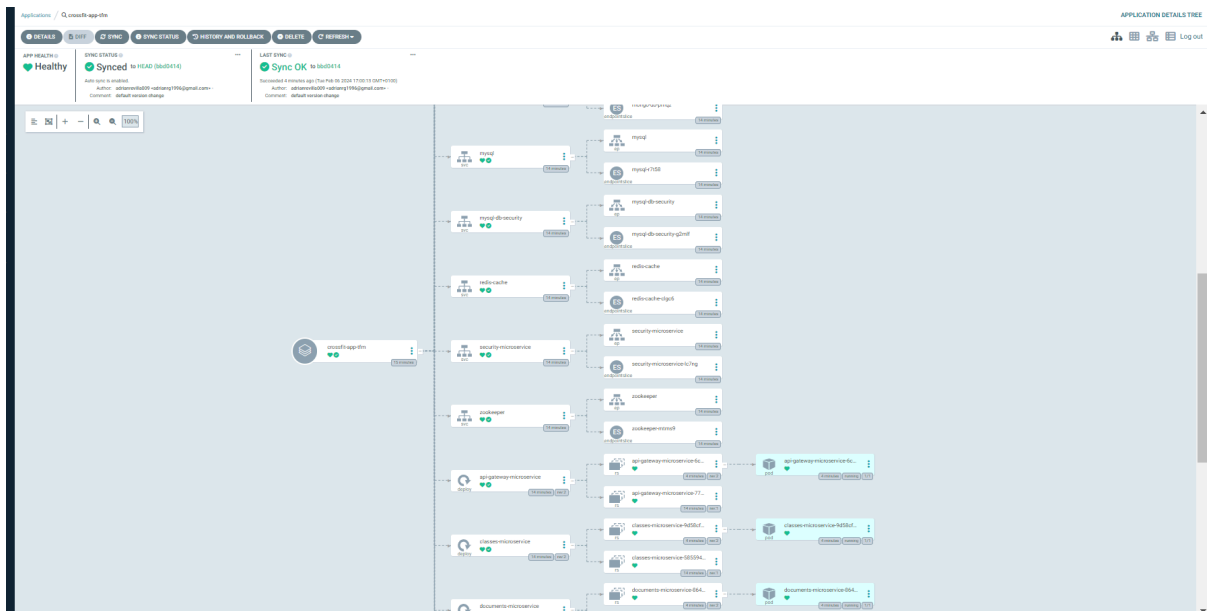
Para ello se definen ciertos manifiestos de configuración que se aplican una vez instalado el chart de helm y que sirven para, mediante unas credenciales, tener acceso y poder detectar esos cambios mencionados y levantar la nueva versión de las imágenes generadas en el entorno de integración continua.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: crossfit-app-tfm
  namespace: argocd
spec:
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: development
  project: default
  source:
    repoURL: 'https://adrianrevilla009:ghp_dW4lxzRCseXMw008TxUcEBz9d4vH732mzCQh@github.com/adrianrevilla009/crossfit-app-TFM.git'
    path: 'helm/app'
    targetRevision: HEAD
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

Esta herramienta proporciona una interfaz gráfica que exponiendo el puerto localmente, podemos ver y comprobar el estado actual de cada contenedor de toda la aplicación.



Monitorización

Como sistema de monitorización se ha escogido el stack grafana, ya que el stack Elasticsearch era demasiado pesado para desplegarlo en un cluster local. Por tanto, se instalan grafana como visor de métricas, prometheus para enviar estas métricas de las aplicaciones al controlador grafana, y loki como sistema de captura de logs para igualmente enviarlos y visualizarlos en grafana.

```

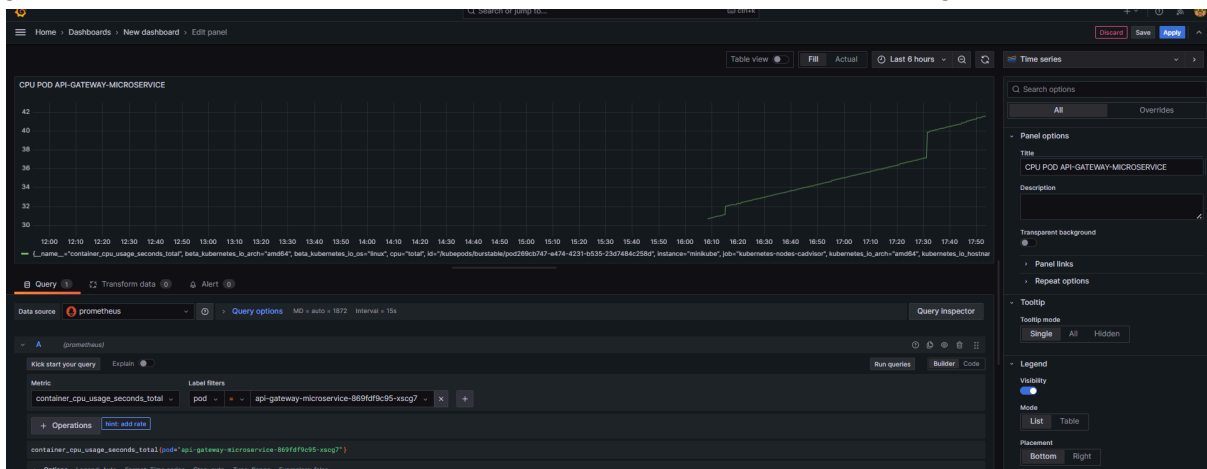
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://loki-prometheus-server.default.svc:80
    isDefault: true

logstash:
  enabled: false

promtail:
  config:
    lokiAddress: http://loki:3100/loki/api/v1/push
  enabled: true
  #nodeSelector:
  #role: nodes-dev-general

```

Al igual que la herramienta anterior, esta también permite exponer hacia fuera una interfaz gráfica que facilita visualizar las métricas recopiladas, tanto a nivel de logs como métricas.



The screenshot shows a Grafana dashboard with a table view of logs. The table has columns for labels, time, line, log, and id. The logs are filtered by 'api-gateway-microservice'. The interface includes a 'Query' section at the bottom with a 'Run query' button and a 'Query options' menu. The right sidebar shows 'Table' settings like 'Panel options', 'Panel title', 'Description', 'Transparent background', 'Panel links', 'Repeat options', 'Table', 'Header', 'Footer', 'Column width', 'Column alignment', 'Cell display mode', 'Call value inspect', 'Column filter', and 'Table footer'.

labels	Time	Line	log	id
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.642	["log":"2024-02-07 14:51:35.825 INFO japi-gateway-microservi...	1707318490642154174	1707318490642154174_40c4d98...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.641	["log":"2024-02-07 14:51:35.825 INFO japi-gateway-microservi...	1707318490641903435	1707318490641903435_2a05599b...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.641	["log":"2024-02-07 14:51:35.766 INFO japi-gateway-microservi...	1707318490641697076	1707318490641697076_94c4e7b...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.641	["log":"2024-02-07 14:51:35.769 INFO japi-gateway-microservi...	1707318490641550150	1707318490641550150_4b03d82...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.641	["log":"2024-02-07 14:51:35.769 INFO japi-gateway-microservi...	1707318490641132173	1707318490641132173_3a67d277...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.640	["log":"2024-02-07 14:51:35.702 INFO japi-gateway-microservi...	1707318490640700579	1707318490640700579_3a6994e...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.640	["log":"2024-02-07 14:51:35.702 INFO japi-gateway-microservi...	1707318490640574017	1707318490640574017_4f49a8c...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.639	["log":"2024-02-07 14:51:35.688 INFO japi-gateway-microservi...	1707318490639977128	1707318490639977128_4f49a8c...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.639	["log":"2024-02-07 14:51:35.641 INFO japi-gateway-microservi...	1707318490639627771	1707318490639627771_50f4b07...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.639	["log":"2024-02-07 14:51:35.640 INFO japi-gateway-microservi...	1707318490639110454	1707318490639110454_Ru0105d...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.638	["log":"2024-02-07 14:51:35.636 INFO japi-gateway-microservi...	1707318490638947342	1707318490638947342_f3a64e4...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.638	["log":"2024-02-07 14:51:35.576 INFO japi-gateway-microservi...	1707318490638713155	1707318490638713155_1a6b8ef...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.638	["log":"2024-02-07 14:51:35.576 INFO japi-gateway-microservi...	1707318490638211493	1707318490638211493_4d0e0d4...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.638	["log":"2024-02-07 14:51:35.573 INFO japi-gateway-microservi...	1707318490638079912	1707318490638079912_d804d831...
("app": "api-gateway-microservice", "container": "...")	2024-02-07 16:08:10.637	["log":"2024-02-07 14:51:35.512 INFO japi-gateway-microservi...	1707318490637766099	1707318490637766099_3cfc8f89...

Testing

Para completar el ciclo de vida del software desarrollado, se ha definido una batería de test unitarios que se ejecutan al pasar el sistema de integración continua.

Además, se ha escogido utilizar un JMeter distribuido para definir una serie de test de carga y así probar la capacidad del cluster para recibir peticiones entrantes y poder visualizar los resultados en el grafana antes mencionado.

```
</elementProp>
<stringProp name="HTTPSampler.domain">api-gateway-microservice.development</stringProp>
<stringProp name="HTTPSampler.port">8083</stringProp>
<stringProp name="HTTPSampler.protocol">http</stringProp>
<stringProp name="HTTPSampler.contentEncoding"></stringProp>
<stringProp name="HTTPSampler.path">/api/classes/</stringProp>
<stringProp name="HTTPSampler.method">GET</stringProp>
<boolProp name="HTTPSampler.follow_redirects">true</boolProp>
```

Para ello, se describen unos archivos de configuración .jmx que serán ejecutados desde el controlador de la herramienta y verificar la tasa de acierto y hasta que punto es capaz de recibir peticiones correctas.

```
PS C:\Projects\master\TFM\crossfit-app-TFM> kubectl exec -i jmeter-distributed-jmeter-master-578cc6544b-s87zt -- jmeter -n -t /jmeter/jmeter-get-classes-test.jmx -R %SERVER_IP%
Creating summariser <summary>
Created the tree successfully using /jmeter/jmeter-get-classes-test.jmx
Configuring remote engine: %SERVER_IP%
Invalid URL String: //%SERVER_IP%/JMeterEngine
0.52 10.244.0.55 10.244.0.54
Creating summariser <summary>
Created the tree successfully using /jmeter/jmeter-get-classes-test.jmx
Configuring remote engine: 10.244.0.52
Configuring remote engine: 10.244.0.55
Configuring remote engine: 10.244.0.54
Configuring remote engine: 10.244.0.54
Starting remote engines
Starting the test @ Mon Apr 01 15:39:47 UTC 2024 (1711985987559)
Remote engines have been started
Waiting for possible Shutdown/StopTestNow/Heapdump message on port 4445
summary = 300 in 00:01:20 = 3.8/s Avg: 794 Min: 545 Max: 2926 Err: 0 (0.00%)
Tidying up remote @ Mon Apr 01 15:41:08 UTC 2024 (171198668412)
... end of run
PS C:\Projects\master\TFM\crossfit-app-TFM>
```

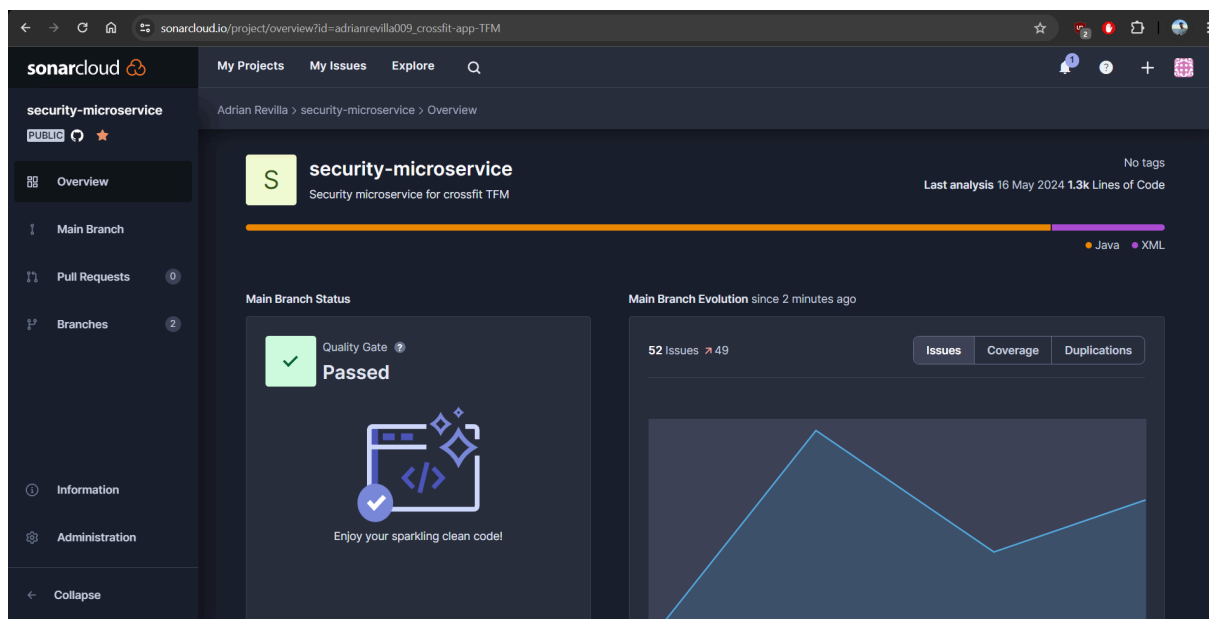
Mantenimiento

SonarCloud ha sido la herramienta elegida para hacer un análisis del código sobre el repositorio github. Se ha enlazado mediante un action en el sistema de CI para que cuando pase los diferentes flujos, analice los 4 repositorios dentro del monorepo que se ha montado.

En la siguiente imagen vemos como pasa el análisis de los 4 proyectos.



En esta otra, vemos ya una vez accedido al sonar, los errores de código que detecta esta herramienta.



De esta forma tenemos una manera de trazar los diferentes bad smells, buenas prácticas etc que nos permite identificar el sonar.

Despliegue

Se ha decidido utilizar Terraform como herramienta de IaC para automatizar el despliegue y simular el entorno productivo en AWS. Dado que desplegar cualquier infraestructura tiene un sobrecoste, el cual iba a ser evidente al desplegar un ELK, 3 bases de datos, un kafka etc, se ha decidido utilizar un minikube dentro de una instancia EC2. Para ello, se despliega esta instancia en la red por defecto y con un grupo de seguridad abriendo los puertos SSH, HTTP y el escogido para exponer el microservicio api-gateway.

```
terraform {
  required_version = ">= 0.13"
  required_providers {
    kubectl = {
      source = "gavinbunney/kubectl"
      version = ">= 1.14.0"
    }
    aws = {
      source = "hashicorp/aws"
      version = ">= 4.53.0"
    }
    time = {
      source = "hashicorp/time"
      version = ">= 0.9.1"
    }
  }
}
```

Se define un script de inicio que levanta el minikube con todos los recursos cuando se levanta la instancia, y que hace un port-forward sobre el puerto que vamos a exponer, y así poder peticionar contra el entorno levantado en el cloud.

```
ubuntu@ip-172-31-2-165:/app$ kubectl get svc -n development
NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
api-gateway-microservice            NodePort    10.109.41.239    <none>           8083:31268/TCP   6m18s
classes-microservice                 ClusterIP    10.105.53.96     <none>           8080/TCP         6m18s
documents-microservice               ClusterIP    10.111.70.49     <none>           8081/TCP         6m18s
kafka-service                       ClusterIP    10.109.193.26    <none>           9092/TCP         6m18s
mongo-db                            ClusterIP    10.109.149.157   <none>           27017/TCP        6m18s
mysql                                ClusterIP    10.105.34.186    <none>           3306/TCP         6m18s
mysql-db-security                   ClusterIP    10.98.136.252    <none>           3306/TCP         6m18s
redis-cache                         ClusterIP    10.105.168.178   <none>           6379/TCP         6m18s
security-microservice                ClusterIP    10.111.153.211   <none>           8082/TCP         6m18s
zookeeper                           ClusterIP    10.106.110.59    <none>           2181/TCP         6m18s

ubuntu@ip-172-31-2-165:~$ kubectl port-forward --address 0.0.0.0 svc/api-gateway-microservice 8080:8083 -n development
Forwarding from 0.0.0.0:8080 -> 8083
Handling connection for 8080
```

En la siguiente imagen vemos como el CI despliega el directorio terraform, donde están todos los archivos de configuración asociados al stack levantado.

```
Summary
Jobs
  build and push
  Terraform Plan & Apply
Run details
  Usage
  Workflow file

Terraform Plan & Apply
Started 1h 25s ago

14 Terraform init, Plan, and Apply
15
16 Initializing the backend...
17
18 Initializing modules...
19
20 - ec2 in ec2
21
22 Initializing provider plugins...
23   - Reusing previous version of hashicorp/aws from the dependency lock file
24   - Reusing previous version of hashicorp/time from the dependency lock file
25   - Reusing previous version of gavinbunney/subctl from the dependency lock file
26   - Installing hashicorp/aws v5.39.0...
27   - Installing hashicorp/time v0.18.0 (signed by hashicorp)
28   - Installing hashicorp/time v0.18.0...
29   - Installing hashicorp/time v0.18.0 (signed by hashicorp)
30   - Installing gavinbunney/subctl v1.14.0...
31   - Installing gavinbunney/subctl v1.14.0 (self-signed, key ID A06421706A00072F)
32   Partner and community providers are signed by their developers.
33   If you'd like to know more about provider signing, you can read about it here:
34   https://www.terraform.io/docs/cli/plugins/signing.html
35   Terraform has made some changes to the provider dependency selections recorded
36   in the .terraform.lock.hcl file. Review those changes and commit them to your
37   version control system if they represent changes you intended to make.
38   Terraform has been successfully initialized!
39
40 You may now begin working with Terraform. Try running "terraform plan" to see
41 any changes that are required for your infrastructure. All Terraform commands
42 should now work.
43
44 If you ever set or change modules or backend configuration for Terraform,
45 rerun this command to reinitialize your working directory. If you forget, other
46 commands will detect it and remind you to do so if necessary.
47
48
49 Warning: The "set-output" command is deprecated and will be disabled soon. Please upgrade to using Environment Files. For more information see: https://github.blog/changelog/2022-10-11-github-actions-deprecating-save-state-and-set-output-commands/
50
51 Warning: The "set-output" command is deprecated and will be disabled soon. Please upgrade to using Environment Files. For more information see: https://github.blog/changelog/2022-10-11-github-actions-deprecating-save-state-and-set-output-commands/
52
53 Warning: The "set-output" command is deprecated and will be disabled soon. Please upgrade to using Environment Files. For more information see: https://github.blog/changelog/2022-10-11-github-actions-deprecating-save-state-and-set-output-commands/
54
55 /home/runner/work/1/tmp/ed82327f-4e24-408f-805f-3887a202f5ab/terraform-bin validate
56 Success! The configuration is valid.
57
58 Warning: The "set-output" command is deprecated and will be disabled soon. Please upgrade to using Environment Files. For more information see: https://github.blog/changelog/2022-10-11-github-actions-deprecating-save-state-and-set-output-commands/
59
60 Warning: The "set-output" command is deprecated and will be disabled soon. Please upgrade to using Environment Files. For more information see: https://github.blog/changelog/2022-10-11-github-actions-deprecating-save-state-and-set-output-commands/
61
62 /home/runner/work/1/tmp/ed82327f-4e24-408f-805f-3887a202f5ab/terraform-bin validate
63 Success! The configuration is valid.
```

Finalmente vemos como podemos peticionar directamente contra la IP pública de la máquina y simular la ejecución de la aplicación en el cloud.

Instancias (1) Información [Refresh] [Conectar] [Estado de la instancia ▼] [Acciones ▼] [Lanzar instancias ▼]

Buscar instancia por atributo o etiqueta (case-sensitive) [En ejecución ▼] < 1 > [Settings]

	Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de	Estado de la al	Zona de dispon...	DNS de IPv4 pú
☐	EC2 Minikube	i-0c546ddc1643ffb57	En ejecución	t2.xlarge	Inicializando	Ver alarmas +	us-west-1a	ec2-13-57-229-

Console

```
POST http://52.53.178.62:8080/api/auth/signup 200 310 ms
POST http://52.53.178.62:8080/api/auth/signin 200 297 ms
GET http://52.53.178.62:8080/api/classes/ 200 1302 ms
```

Conclusiones

Dada la ambición del proyecto e intentar abarcar una aplicación lo más asemejable posible a lo que se puede encontrar hoy en día en la actualidad, se ha llevado hasta los límites posibles.

Una posible mejora podría haber sido el uso de un ingress y monitorizar sobre ello, también se podría haber securizado el cluster con alguna malla de red como istio, o incluso haber metido algún manejador de secretos como vault.

La idea principal era montar la infraestructura con sus respectivos componentes en algún cloud con un EKS y un terraform para automatizarlo, pero dado que ninguna infraestructura es gratuita, se ha montado de la forma menos costosa posible.

Por tanto, y como conclusión final, decir que se ha intentado cubrir todas competencias tanto de desarrollo, arquitectura e infraestructura que abarcan una aplicación.

Bibliografía

- Spring gateway: <https://spring.io/projects/spring-cloud-gateway>
- String kafka: <https://spring.io/projects/spring-kafka>
- Spring redis: <https://spring.io/projects/spring-data-redis>
- Spring mongodb: <https://spring.io/projects/spring-data-mongodb>
- Spring mysql: <https://spring.io/guides/gs/accessing-data-mysql>
- Arquitectura hexagonal: <https://medium.com/ssense-tech/hexagonal-architecture-there-are-always-two-sides-to-every-story-bc0780ed7d9c>
- Spring swagger: <https://www.baeldung.com/swagger-2-documentation-for-spring-rest-api>
- Spring prometheus: <https://medium.com/simform-engineering/revolutionize-monitoring-empowering-spring-boot-applications-with-prometheus-and-grafana-e99c5c7248cf>
- Spring loki: <https://www.baeldung.com/spring-boot-loki-grafana-logging>
- Spring docker dependencias: <https://spring.io/guides/topicals/spring-boot-docker>
- Spring security: <https://spring.io/projects/spring-security>
- Dockerfiles: <https://docs.docker.com/reference/dockerfile/>
- Helm: <https://helm.sh/>
- Bash: <https://devhints.io/bash>
- Kubernetes: <https://kubernetes.io/es/docs/home/>
- Redes en k8s: <https://kubernetes.io/docs/concepts/services-networking/ingress/>
- Redis en k8s: <https://redis.io/docs/latest/integrate/redis-data-integration/installation/install-k8s/>
- Kafka en k8s: <https://medium.com/@mustafaguc/how-to-easily-deploy-apache-kafka-on-kubernetes-or-locally-3375cc9d2015>
- Mongo en k8s: <https://www.mongodb.com/products/integrations/kubernetes>
- Volumen persistentes en k8s: <https://kubernetes.io/es/docs/concepts/storage/persistent-volumes/>
- Jmeter: <https://jmeter.apache.org/>
- Jmeter en k8s: <https://faun.pub/ultimate-jmeter-kubernetes-starter-kit-7eb1a823649b>
- Grafana: <https://grafana.com/>
- Grafana on k8s: <https://grafana.com/docs/grafana/latest/setup-grafana/installation/kubernetes/>
- Prometheus: <https://prometheus.io/>
- Prometheus en k8s: <https://alexmarket.medium.com/monitoreando-un-cluster-de-kubernetes-con-prometheus-y-grafana-4c253c1631b4>
- Loki: <https://grafana.com/oss/loki/>
- Loki en k8s: <https://grafana.com/docs/loki/latest/setup/install/helm/>
- Sonarcloud: <https://sonarcloud.io/login>
- Github actions: <https://github.com/features/actions>
- Argo cd: <https://argo-cd.readthedocs.io/en/stable/>
- Argo cd en k8s: <https://argo-cd.readthedocs.io/en/stable/operator-manual/installation/>

- Dependencias en container registry:
<https://docs.github.com/es/packages/working-with-a-github-packages-registry/working-with-the-container-registry>
- Terraform: <https://www.terraform.io/>
- AWS: <https://aws.amazon.com/es/>

Anexos

Proyecto principal en github: <https://github.com/adrianrevilla009/crossfit-app-TFM>

Biblioteca común en github: <https://github.com/adrianrevilla009/common-TFM>